딥러닝 기반 저선량 CT 영상 복원 및 잔차 연결·오류 원인 심층 분석

Low-Dose CT Image Reconstruction with U-Net Variants: Original Raw DICOM ($[-100, 300]$ HU) Pipeline, Residual Learning, and Error Source Analysis

교과목: AI for Medical Image Analysis (성균관대학교 메타바이오헬스학과) | **학번·성명:** StudentID Name | **제출일:** 2026. 11. 08.

[핵심 요약 및 추가 가산점(Extra Credit) 충족 사항]

- **원본 Raw DICOM 데이터 및 $[-100, 300]$ HU 윈도우 적용 (Extra Credit 요건 충족):** 리사이즈된 256×256 PNG 대신 원본 512×512 `.IMA` **Raw** 데이터(`3mm` slice thickness, `Sharp` kernel, 총 10명 2378장 쌍)를 직접 처리하였으며, 모든 시각적 복원 결과를 $[-100, 300]$ **HU** 물리적 감쇠 범위로 역변환하여 제시함. 테스트 환자(`L192`, `L506`, 총 451장)는 학습·검증 과정에서 완전히 분리함.
- **다중 아키텍처·증강 실험 및 잔차 연결(Residual Connection) 비교:** 5종의 U-Net 계열 네트워크(`U_Net`, `R2U_Net`, `AttU_Net`, `R2AttU_Net`, `NestedUNet`)와 4종의 MONAI 데이터 증강 조합을 설계함. 전역 잔차 연결($\hat{y} = \text{clamp}(x + f_\theta(x), 0, 1)$)의 이론적·실증적 효과와 블록 내부 잔차 연결 유무(`U_Net` vs. `R2U_Net`)를 정량·정성적으로 비교함.
- **핵심 정량 성과 및 심층 오류 분석:** 평가 전용 테스트셋(`L192`, `L506`) 기준 Quarter-Dose 입력 대비 최고 모델(`U_Net_none`)이 **PSNR 21.75 dB → 27.17 dB (+5.43 dB)**, **SSIM 0.7160 → 0.8344 (+0.1184)**, **RMSE 33.45 HU → 17.80 HU**의 유의미한 복원 성능 향상을 달성함. 나아가 FP16 혼합 정밀도(`AMP`) + `SSIMLoss` 분산 계산의 부동소수점 수치 불안정성(`NaN`)과 쌍(Paired) 증강의 타깃 오염 원인을 수식·실험적으로 규명하고 에포크 체크포인트 복구 기법으로 검증함.

1. 서론 및 과제 개요 (Introduction & Problem Formulation)

컴퓨터 단층촬영(Computed Tomography, CT)은 흉·복부 장기의 해부학적 미세 구조와 병변을 비침습적으로 진단하는 핵심 영상 기법이나, X선 방사선 피폭 위험으로 인해 임상에서는 ALARA (As Low As Reasonably Achievable) 원칙에 따라 방사선량을 낮춘 저선량 CT (Low-Dose CT, LDCT) 촬영이 요구된다. 그러나 관전류·노출량을 1/4 수준(Quarter Dose)으로 감소시키면 검출기에 도달하는 광자 수 부족(Photon Starvation)으로 인해 투영(Sinogram) 도메인에서 포아송 양자 잡음(Poisson Quantum Mottle)과 전자 잡음이 급증하며, 역투영 재구성 후 영상 전반에 입상 잡음(Granular Noise)과 줄무늬 아티팩트(Streak Artifact)가 발생한다. 특히 본 과제의 대상인 **3mm 슬라이스 두께 및 Sharp 재구성 커널(Sharp Reconstruction Kernel)** 영상은 장기 경계와 혈관의 고주파 공간 해상도를 극대화하는 대신 고주파 양자 잡음까지 함께 증폭시키므로, **세부 해부학적 경계(Edge)를 뭉개지 않으면서(Over-smoothing 방지) 선택적으로 잡음만 억제**하는 것이 핵심 난제이다.

본 프로젝트에서는 저선량 Quarter-Dose CT 영상 $x \in [0, 1]^{1 \times 512 \times 512}$로부터 정상 선량 Full-Dose CT 영상 $y \in [0, 1]^{1 \times 512 \times 512}$을 복원하는 역문제(Inverse Problem)를 풀기 위해 PyTorch 및 MONAI 기반의 엔드투엔드 딥러닝 파이프라인을 구축하였다. 단순 단일 모델 학습에 그치지 않고, (1) 공간 해상도 보존 패딩(`padding=1`)과 전역 잔차 학습(Global Residual Learning)을 결합한 5종의 U-Net 변형 아키텍처 비교, (2) 블록 내부 잔차 연결(`RRCNN_block`) 유무에 따른 수렴성·경계 보존성 비교, (3) 4종의 기하·강도 데이터 증강 실험 및 혼합 정밀도 학습 과정에서의 수치적 오류 원인(Source of Errors) 분석을 체계적으로 수행하였다.

2. 데이터셋 구성 및 전처리 파이프라인 (Dataset & $[-100, 300]$ HU Preprocessing)

2.1 원본 Raw DICOM (`.IMA`) 데이터셋 구성 및 환자 분리 전략

과제 지침(Week 9 Slide 19)의 추가 가산점 요건에 맞춰, 사전 리사이즈된 256×256 PNG 파일 대신 Kaggle (2016 NIH-AAPM-Mayo Clinic Low Dose CT Grand Challenge 기반)의 **원본 Raw DICOM (`.IMA`) 데이터**(`full_3mm_sharp` 및 `quarter_3mm_sharp`, 해상도 512×512)를 직접 사용하였다. 데이터 누수(Data Leakage)를 엄격히 차단하기 위해 경로 필터링을 통해 **`L192` (240장)와 `L506` (211장), 총 451장을 학습에서 완전히 배제하여 독립 테스트셋(Held-out Test Set)**으로 고정하였다. 나머지 8명 환자의 슬라이스 쌍(총 1927장)은 고정 시드(`seed=42`) 기반 `ShuffleSplit`을 통해 학습용 1541장(80%)과 검증용 386장(20%)으로 분할하였다(표 1).

표 1: 원본 Raw DICOM (.IMA, 3mm Sharp Kernel) 환자별 슬라이스 분포 및 학습·검증·테스트 분할 요약

Subject ID	Reconstruction Series	Slices (Pairs)	Resolution	Dataset Split Role
L067	full/quarter_3mm_sharp	224	512×512	Train / Val Pool (80:20 분할)
L096	full/quarter_3mm_sharp	330	512×512	Train / Val Pool (80:20 분할)
L109	full/quarter_3mm_sharp	128	512×512	Train / Val Pool (80:20 분할)
L143	full/quarter_3mm_sharp	234	512×512	Train / Val Pool (80:20 분할)
L192	full/quarter_3mm_sharp	240	512×512	**Held-out Test (평가 전용)**
L286	full/quarter_3mm_sharp	210	512×512	Train / Val Pool (80:20 분할)
L291	full/quarter_3mm_sharp	343	512×512	Train / Val Pool (80:20 분할)
L310	full/quarter_3mm_sharp	214	512×512	Train / Val Pool (80:20 분할)
L333	full/quarter_3mm_sharp	244	512×512	Train / Val Pool (80:20 분할)
L506	full/quarter_3mm_sharp	211	512×512	**Held-out Test (평가 전용)**
Total (10 Subjects)	3mm Slice, Sharp Kernel (.IMA)	**2378**	$[-100, 300]$ HU	Train 1541 / Val 386 / Test 451

2.2 $[-100, 300]$ HU 윈도잉 전처리 및 데이터 증강 레지스트리 구현

CT 원본 픽셀값은 공기(-1000 HU), 지방($-100 \sim -50$ HU), 물(0 HU), 연조직·간·신장 실질($+30 \sim +150$ HU), 조영 증강 혈관·치밀골($+200 \sim +1000$ HU)에 걸쳐 넓은 동적 범위를 가진다. 전체 HU 범위를 단순 선형 정규화할 경우 복부 진단의 핵심인 연조직 및 혈관 내 미세 대조도(Contrast)가 압축되어 신경망의 유효 비트 해상도가 손실된다. 이에 따라 `src/data.py`(그림 1a)에서는 MONAI의 `ITKReader`로 원본 `.IMA`를 로드한 뒤, `ScaleIntensityRanged(a_min=-100.0, a_max=300.0, b_min=0.0, b_max=1.0, clip=True)`를 적용하여 $[-100, 300]$ HU 구간을 $[0, 1]$로 매핑하였다. 보고서의 모든 시각적 결과는 역변환식 $I_{\text{HU}} = 400 \cdot \hat{y} - 100$을 통해 물리적 $[-100, 300]$ HU 스케일로 복원하여 표시한다. 또한 `src/utils.py`(그림 1b)에는 입력(`quarter`)과 정답(`full`)에 동일한 랜덤 시드로 동기화된 변환을 수행하는 Dictionary 기반 증강 레지스트리(`none`, `RandAffined`, `Affine_Noise_Combo`, `Rot90_Scale_Combo`)를 구현하였다.

3. 네트워크 아키텍처, 손실 함수 및 학습 프레임워크 구현 (Code Implementation)

3.1 해상도 보존 U-Net 계열 5종 아키텍처 설계

과제 요구사항(Slide 20)에 따라 모든 합성곱 계층에서 공간 해상도가 손실되지 않도록 3×3 커널에 `stride=1`, `padding=1`을 적용하였다(`src/UNet/Models.py`, 그림 2). 인코더는 4단계 `MaxPool2d(2, 2)`를 통해 채널 수를 $64 \rightarrow 128 \rightarrow 256 \rightarrow 512 \rightarrow 1024$로 확장하며($512^2 \rightarrow 32^2$), 디코더는 전치 합성곱(Transposed Conv)에서 흔히 발생하는 체커보드 아티팩트(Checkerboard Artifact)를 방지하기 위해 `Upsample(scale_factor=2)` 후 3×3 합성곱(`padding=1`)을 수행하는 `up_conv` 구조를 채택하였다. 마지막 1×1 합성곱 출력에는 입력 영상을 더하는 전역 잔차 연결(`out = self.Conv(d2) + x`)과 물리적 윈도우 경계를 보장하는 `out.clamp(0, 1)`을 결합하였다. 비교 실험을 위해 구축한 5종 아키텍처의 구조적 특성과 연산량(512×512 단일 슬라이스 기준)은 표 2와 같다.

표 2: 구현된 5종 U-Net 계열 복원 네트워크의 구조적 특징, 파라미터 수 및 연산 복잡도(512×512 입력 기준)

Architecture	Block Res.	Global $+x$	Recurrent (t=2)	Att. Gate	Nested Skip	Params	GFLOPs (512^2)
`U_Net` (U-Net (Global +x))	—	✓	—	—	—	34.53 M	522.2
`U_Net_NoRes` (U-Net (No Global +x))	—	—	—	—	—	34.53 M	522.2
`R2U_Net` (R2U-Net)	✓	✓	✓	—	—	39.09 M	1219.8
`AttU_Net` (Attention U-Net)	—	✓	—	✓	—	34.88 M	530.8
`R2AttU_Net` (R2Attention U-Net)	✓	✓	✓	✓	—	39.44 M	1228.5
`NestedUNet` (UNet++ (Nested U-Net))	—	✓	—	—	✓	36.63 M	1102.0

3.2 복합 손실 함수($L_1 + 0.1 \cdot \mathcal{L}_{\text{SSIM}}$), 평가 지표 및 학습 엔진 구현

영상 복원에서 L_2 (MSE) 손실만을 최소화하면 조건부 사후 분포의 평균을 예측하려는 성질로 인해 고주파 질감이 흐려지는 과도한 평활화(Over-smoothing)가 발생한다. 이를 극복하기 위해 `src/main.py`에서는 이상치에 강건하고 경계 선명도를 유지하는 픽셀 단위 L_1 손실과 국소 휘도·대조도·구조적 유사성을 직접 최적화하는 $\mathcal{L}_{\text{SSIM}} = 1 - \text{SSIM}(\hat{y}, y)$를 결합한 복합 손실 함수를 설계하였다:

$$\mathcal{L}(\hat{y}, y) = \|\hat{y} - y\|_1 + 0.1 \cdot \big(1 - \text{SSIM}(\hat{y}, y)\big) \tag{1}$$

1. Raw DICOM Loading, [-100, 300] HU Windowing & Subject-Disjoint Split

src/data.py (L19-L41, L50-L104)

```python
def get_augment(augment: list | None):
    base_transforms = [
        LoadImaged(keys=["quarter", "full"], reader="ITKReader"),
        EnsureChannelFirstd(keys=["quarter", "full"]),
        SqueezeDimd(keys=["quarter", "full"], dim=-1),
        ScaleIntensityRanged(
            keys=["quarter", "full"],
            a_min=-100.0,
            a_max=300.0,
            b_min=0.0,
            b_max=1.0,
            clip=True,
        ),
        CastToTyped(keys=["quarter", "full"], dtype=torch.float32),
    ]

    if augment is not None:
        transforms = Compose(base_transforms + augment)
    else:
        transforms = Compose(base_transforms)

    return transforms

  (intermediate lines omitted for brevity)
def data_load(
    target_dir: str | Path = ROOT / "Full Dose/",
    input_dir: str | Path = ROOT / "Quarter Dose/",
    augment: list | None = None,
    seed: int = 42,
):
    train_transforms = get_augment(augment)
    test_transforms = get_augment(None)

    target_path = sorted(list(Path(target_dir).rglob("*.IMA")))
    input_path = sorted(list(Path(input_dir).rglob("*.IMA")))

    # 2 data for test set: L192 and L506.
    full_path = [
        str(path)
        for path in target_path
        if any("L192" in p or "L506" in p for p in path.parts)
    ]
    quarter_path = [
        str(path)
        for path in input_path
        if any("L192" in p or "L506" in p for p in path.parts)
    ]

    test_path = [
        {"quarter": quarter, "full": full}
        for full, quarter in zip(full_path, quarter_path)
    ]

    test_set = Dataset(data=test_path, transform=test_transforms)

    # Others: train & val
    full_path = [
        str(path)
        for path in target_path
        if not any("L192" in p or "L506" in p for p in path.parts)
    ]
    quarter_path = [
        str(path)
        for path in input_path
        if not any("L192" in p or "L506" in p for p in path.parts)
    ]
    learnable_path = np.array(
        [
            {"quarter": quarter, "full": full}
            for full, quarter in zip(full_path, quarter_path)
        ]
    )

    ss = ShuffleSplit(n_splits=1, test_size=0.2, train_size=0.8, random_state=seed)
    train_index, val_index = next(iter(ss.split(X=full_path)))
    train_set = Dataset(data=learnable_path[train_index], transform=train_transforms)
    val_set = Dataset(data=learnable_path[val_index], transform=test_transforms)

    return train_set, val_set, test_set
```

(a) `src/data.py`: $[-100, 300]$ HU 정규화 및 `L192`/`L506` 분리

2. MONAI Paired Data Augmentation Registry (Single & Composite Transforms)

src/utils.py (L20-L36, L45-L61, L72-L103)

```python
    definitions = {
        "RandAffined": {
            "cls": getattr(transforms_module, "RandAffined", None),
            "kwargs": {
                "keys": ["quarter", "full"],
                "rotate_range": (-0.17, 0.17),
                "translate_range": (4, 4),
                "scale_range": (0.1, 0.1),
                "prob": 0.5,
                "padding_mode": "reflection",
                "mode": "bilinear",
            },
        },
        "RandRot90d": {
            "cls": getattr(transforms_module, "RandRot90d", None),
            "kwargs": {"keys": ["quarter", "full"], "prob": 0.5, "max_k": 3},
        },
  (intermediate lines omitted for brevity)
        "RandScaleIntensityd": {
            "cls": getattr(transforms_module, "RandScaleIntensityd", None),
            "kwargs": {"keys": ["quarter", "full"], "factors": 0.1, "prob": 0.5},
        },
        "RandShiftIntensityd": {
            "cls": getattr(transforms_module, "RandShiftIntensityd", None),
            "kwargs": {"keys": ["quarter", "full"], "offsets": 0.1, "prob": 0.5},
        },
        "RandGaussianNoised": {
            "cls": getattr(transforms_module, "RandGaussianNoised", None),
            "kwargs": {
                "keys": ["quarter", "full"],
                "mean": 0.0,
                "std": 0.05,
                "prob": 0.3,
            },
        },
  (intermediate lines omitted for brevity)
        "Affine_Noise_Combo": {
            "composite": [
                (
                    "RandAffined",
                    {
                        "keys": ["quarter", "full"],
                        "rotate_range": (-0.17, 0.17),
                        "prob": 0.5,
                        "mode": "bilinear",
                    },
                ),
                (
                    "RandGaussianNoised",
                    {
                        "keys": ["quarter", "full"],
                        "mean": 0.0,
                        "std": 0.05,
                        "prob": 0.3,
                    },
                ),
            ]
        },
        "Rot90_Scale_Combo": {
            "composite": [
                ("RandRot90d", {"keys": ["quarter", "full"], "prob": 0.5, "max_k": 3}),
                (
                    "RandScaleIntensityd",
                    {"keys": ["quarter", "full"], "factors": 0.1, "prob": 0.5},
                ),
            ]
        },
    }
```

(b) `src/utils.py`: MONAI 기반 단일·복합 데이터 증강 레지스트리

그림 1: 원본 Raw DICOM 데이터 로딩·전처리 및 증강 파이프라인 핵심 구현 코드 캡처

3. U-Net Without Global Residual (U_Net_NoRes) vs. Global +x (U_Net) & R2U-Net

src/UNet/Models.py (L5-L25, L47-L54, L115-L124, L194-L203, L205-L247)

```python
class conv_block(nn.Module):
    """
    Convolution Block
    """

    def __init__(self, in_ch, out_ch):
        super(conv_block, self).__init__()

        self.conv = nn.Sequential(
            nn.Conv2d(in_ch, out_ch, kernel_size=3, stride=1, padding=1, bias=True),
            nn.BatchNorm2d(out_ch),
            nn.ReLU(inplace=True),
            nn.Conv2d(out_ch, out_ch, kernel_size=3, stride=1, padding=1, bias=True),
            nn.BatchNorm2d(out_ch),
            nn.ReLU(inplace=True),
        )

    def forward(self, x):

        x = self.conv(x)
        return x
    (intermediate lines omitted for brevity)
class U_Net_NoRes(nn.Module):
    """
    UNet - Basic Implementation
    Paper : https://arxiv.org/abs/1505.04597
    """

    def __init__(self, in_ch=3, out_ch=1):
        super(U_Net_NoRes, self).__init__()
    (intermediate lines omitted for brevity)
        d2 = self.Up2(d3)
        d2 = torch.cat((e1, d2), dim=1)
        d2 = self.Up_conv2(d2)

        out = self.Conv(d2)  # + x

        # d1 = self.active(out)

        return out.clamp(0, 1)

    (intermediate lines omitted for brevity)
        d2 = self.Up2(d3)
        d2 = torch.cat((e1, d2), dim=1)
        d2 = self.Up_conv2(d2)

        out = self.Conv(d2) + x

        # d1 = self.active(out)

        return out.clamp(0, 1)

    (intermediate lines omitted for brevity)
class Recurrent_block(nn.Module):
    """
    Recurrent Block for R2Unet_CNN
    """

    def __init__(self, out_ch, t=2):
        super(Recurrent_block, self).__init__()

        self.t = t
        self.out_ch = out_ch
        self.conv = nn.Sequential(
            nn.Conv2d(out_ch, out_ch, kernel_size=3, stride=1, padding=1, bias=True),
            nn.BatchNorm2d(out_ch),
            nn.ReLU(inplace=True),
        )

    def forward(self, x):
        for i in range(self.t):
            if i == 0:
                x = self.conv(x)
            out = self.conv(x + x)
        return out


class RRCNN_block(nn.Module):
    """
    Recurrent Residual Convolutional Neural Network Block
    """

    def __init__(self, in_ch, out_ch, t=2):
        super(RRCNN_block, self).__init__()

        self.RCNN = nn.Sequential(
            Recurrent_block(out_ch, t=t), Recurrent_block(out_ch, t=t)
        )
        self.Conv = nn.Conv2d(in_ch, out_ch, kernel_size=1, stride=1, padding=0)

    def forward(self, x):
        x1 = self.Conv(x)
        x2 = self.RCNN(x1)
        out = x1 + x2
        return out
```

(a) `U_Net(conv_block)`과 `R2U_Net(RRCNN_block)` 비교

4. Attention Gate (AttU-Net / R2AttU-Net) & Dense Nested Skips (UNet++)

src/UNet/Models.py (L333-L365, L626-L648)

```python
class Attention_block(nn.Module):
    """
    Attention Block
    """

    def __init__(self, F_g, F_l, F_int):
        super(Attention_block, self).__init__()

        self.W_g = nn.Sequential(
            nn.Conv2d(F_l, F_int, kernel_size=1, stride=1, padding=0, bias=True),
            nn.BatchNorm2d(F_int),
        )

        self.W_x = nn.Sequential(
            nn.Conv2d(F_g, F_int, kernel_size=1, stride=1, padding=0, bias=True),
            nn.BatchNorm2d(F_int),
        )

        self.psi = nn.Sequential(
            nn.Conv2d(F_int, 1, kernel_size=1, stride=1, padding=0, bias=True),
            nn.BatchNorm2d(1),
            nn.Sigmoid(),
        )

        self.relu = nn.ReLU(inplace=True)

    def forward(self, g, x):
        g1 = self.W_g(g)
        x1 = self.W_x(x)
        psi = self.relu(g1 + x1)
        psi = self.psi(psi)
        out = x * psi
        return out
    (intermediate lines omitted for brevity)
    def forward(self, x):

        x0_0 = self.conv0_0(x)
        x1_0 = self.conv1_0(self.pool(x0_0))
        x0_1 = self.conv0_1(torch.cat([x0_0, self.Up(x1_0)], 1))

        x2_0 = self.conv2_0(self.pool(x1_0))
        x1_1 = self.conv1_1(torch.cat([x1_0, self.Up(x2_0)], 1))
        x0_2 = self.conv0_2(torch.cat([x0_0, x0_1, self.Up(x1_1)], 1))

        x3_0 = self.conv3_0(self.pool(x2_0))
        x2_1 = self.conv2_1(torch.cat([x2_0, self.Up(x3_0)], 1))
        x1_2 = self.conv1_2(torch.cat([x1_0, x1_1, self.Up(x2_1)], 1))
        x0_3 = self.conv0_3(torch.cat([x0_0, x0_1, x0_2, self.Up(x1_2)], 1))

        x4_0 = self.conv4_0(self.pool(x3_0))
        x3_1 = self.conv3_1(torch.cat([x3_0, self.Up(x4_0)], 1))
        x2_2 = self.conv2_2(torch.cat([x2_0, x2_1, self.Up(x3_1)], 1))
        x1_3 = self.conv1_3(torch.cat([x1_0, x1_1, x1_2, self.Up(x2_2)], 1))
        x0_4 = self.conv0_4(torch.cat([x0_0, x0_1, x0_2, x0_3, self.Up(x1_3)], 1))

        output = self.final(x0_4) + x
        return output.clamp(0, 1)
```

(b) `Attention_block` 및 `NestedUNet` 밀집 스킵 경로

그림 2: 공간 해상도 보존(`padding=1`), 전역 잔차 연결(`+ x`) 및 변형 U-Net 블록 구현 코드 캡처

또한 과제 요구사항(Slide 21–22)에 맞춰 `src/loss.py`(그림 3a)에 PyTorch Ignite `Metric`을 상속한 커스텀 `PSNR`(max $= 1.0$)과 11×11 가우시안 커널($\sigma = 1.5, C_1 = 0.01^2, C_2 = 0.03^2$) 기반의 2D/3D 호환 `SSIM` 클래스를 직접 구현하였다. 학습 엔진(`src/trainer.py`, 그림 3b)은 MONAI `SupervisedTrainer`를 기반으로 `AdamW`(lr $= 2\times10^{-5}$), `CosineAnnealingLR`, 그래디언트 클리핑(`max_grad_norm=0.1`), 혼합 정밀도(`amp=True`)를 적용하였으며, 10 에포크 주기로 검증 `val_psnr`을 평가하여 조기 종료(`patience=2`) 및 최적 가중치(`best_model.pth`) 기반 자동 테스트 평가를 수행하도록 모듈화하였다.

5. Custom PSNR / Gaussian-Window SSIM Metrics & Composite Loss Function

`src/loss.py` (L41-L51, L119-L161)

```python
    def _compute_psnr(self, y_pred: torch.Tensor, y: torch.Tensor) -> torch.Tensor:
        if not isinstance(y_pred, torch.Tensor):
            y_pred = torch.tensor(y_pred, device=y.device if isinstance(y, torch.Tensor) else None)
        if not isinstance(y, torch.Tensor):
            y = torch.tensor(y, device=y_pred.device)

        mse = torch.mean((y_pred - y) ** 2, dim=list(range(1, y_pred.ndim)))
        max_v = torch.tensor(self.max_val, dtype=y_pred.dtype, device=y_pred.device)
        psnr = 20 * torch.log10(max_v) - 10 * torch.log10(mse + 1e-8)
        return psnr

    (intermediate lines omitted for brevity)
    def _compute_ssim(self, y_pred: torch.Tensor, y: torch.Tensor) -> torch.Tensor:
        if not isinstance(y_pred, torch.Tensor):
            y_pred = torch.tensor(y_pred, device=y.device if isinstance(y, torch.Tensor) else None)
        if not isinstance(y, torch.Tensor):
            y = torch.tensor(y, device=y_pred.device)

        b, c = y_pred.shape[:2]
        device = y_pred.device
        dtype = y_pred.dtype

        C1 = (0.01 * self.max_val) ** 2
        C2 = (0.03 * self.max_val) ** 2

        kernel = self._gaussian_kernel(c, device, dtype)

        if self.spatial_dims == 2:
            conv = lambda x: F.conv2d(
                x, kernel, padding=self.kernel_size[0] // 2, groups=c
            )
        elif self.spatial_dims == 3:
            conv = lambda x: F.conv3d(
                x, kernel, padding=self.kernel_size[0] // 2, groups=c
            )
        else:
            raise ValueError("Only 2D and 3D spatial dimensions are supported.")

        mu1 = conv(y_pred)
        mu2 = conv(y)

        mu1_sq = mu1**2
        mu2_sq = mu2**2
        mu1_mu2 = mu1 * mu2

        sigma1_sq = conv(y_pred**2) - mu1_sq
        sigma2_sq = conv(y**2) - mu2_sq
        sigma12 = conv(y_pred * y) - mu1_mu2

        numerator = (2 * mu1_mu2 + C1) * (2 * sigma12 + C2)
        denominator = (mu1_sq + mu2_sq + C1) * (sigma1_sq + sigma2_sq + C2)

        ssim_map = numerator / (denominator + 1e-8)
        return ssim_map.mean(dim=list(range(2, ssim_map.ndim)))
```

`src/main.py` (L78-L88)

```python
    network.compile()

    # 7. 옵티마이저 및 손실 함수(Loss) 설정
    max_epochs = getattr(args, "epochs", 10)
    lr = getattr(args, "lr", 1e-4)
    optimizer = optim.AdamW(network.parameters(), lr=lr)
    scheduler = optim.lr_scheduler.CosineAnnealingLR(
        optimizer=optimizer, T_max=max_epochs // 10, eta_min=1e-6
    )
    loss_function = lambda x, y: (
        nn.L1Loss()(x, y) + 0.1 * SSIMLoss(spatial_dims=2, data_range=1.0)(x, y)
```

(a) `src/loss.py` & `main.py`: PSNR, SSIM 및 복합 손실 함수

6. MONAI SupervisedTrainer (AMP, Grad Clipping, Early Stopping & Test Eval)

`src/trainer.py` (L96-L111, L166-L231, L383-L407)

```python
        self.trainer = SupervisedTrainer(
            device=self.device,
            max_epochs=self.max_epochs,
            network=self.network,
            train_data_loader=self.train_loader,
            optimizer=self.optimizer,
            loss_function=self.loss_function,
            train_handlers=self.train_handlers,
            prepare_batch=self._custom_prepare_batch,
            amp=True,
        )

        @self.trainer.on("BACKWARD_COMPLETED")
        def grad_clip_handler(engine):
            clip_grad_norm_(self.network.parameters(), max_norm=self.max_grad_norm)

    (intermediate lines omitted for brevity)
                if current_epoch % 10 == 0:
                    print(f"\n[Epoch {current_epoch}] Running validation...")
                    self.evaluator.state.epoch = current_epoch
                    self.evaluator.state.iteration = current_iteration

                    val_metrics = self.eval()
                    print(f"[Epoch {current_epoch}] Validation Complete")

                    for metric_name, metric_value in val_metrics.items():
                        if isinstance(metric_value, torch.Tensor):
                            metric_value = metric_value.item()
                        self.writer.add_scalar(
                            metric_name, metric_value, current_iteration
                        )
                    self.writer.flush()

                    # 스케줄러 업데이트
                    if self.lr_scheduler is not None:
                        if isinstance(
                            self.lr_scheduler,
                            torch.optim.lr_scheduler.ReduceLROnPlateau,
                        ):
                            # val_loss가 없으면 val_psnr을 대체 시용
                            target_metric = (
                                "val_loss" if "val_loss" in val_metrics else "val_psnr"
                            )
                            val_score = val_metrics.get(target_metric, 0.0)
                            if isinstance(val_score, torch.Tensor):
                                val_score = val_score.item()
                            self.lr_scheduler.step(val_score)
                        else:
                            self.lr_scheduler.step()

                    # 얼리스타핑 지표 설정 (설정한 mode의 지표가 없을 경우 대비 fallback 추가)
                    if self.mode == "min":
                        current_score = val_metrics.get(
                            "val_loss", val_metrics.get("val_psnr", 0.0)
                        )
                    else:
                        current_score = val_metrics.get(
                            "val_psnr", val_metrics.get("val_loss", 0.0)
                        )

                    if isinstance(current_score, torch.Tensor):
                        current_score = current_score.item()

                    is_better = (
                        (current_score < self.best_score - self.min_delta)
                        if self.mode == "min"
                        else (current_score > self.best_score + self.min_delta)
                    )

                    if is_better:
                        self.best_score = current_score
                        self.patience_counter = 0
                        self._save_best_model(current_epoch)
                    else:
                        self.patience_counter += 1
                        print(
                            f"[-] Validation score did not improve. Patience: {self.patience_counter}/{self.patience}"
                        )

                    if self.patience_counter >= self.patience:
                        print(f"[!] Early stopping triggered at epoch {current_epoch}!")
                        engine.terminate()

    (intermediate lines omitted for brevity)
        print(
            f"\n[*] Loading best model weights from {self.best_model_path} for testing..."
        )
        checkpoint = torch.load(self.best_model_path, map_location=self.device)
        self.network.load_state_dict(checkpoint["model_state_dict"])
        self.network.eval()

        test_key_metric, test_additional_metrics = self._create_metrics(prefix="test")

        test_evaluator = SupervisedEvaluator(
            device=self.device,
            val_data_loader=self.test_loader,
            network=self.network,
            val_handlers=[],
            key_val_metric=test_key_metric,
            additional_metrics=test_additional_metrics,
            prepare_batch=self._custom_prepare_batch,
            amp=True,
        )

        print("[*] Test evaluation started...")
        test_evaluator.run()
        test_metrics = test_evaluator.state.metrics
        print(f"[*] Test Metrics Result: {test_metrics}")
```

(b) `src/trainer.py`: AMP, Grad Clipping, 조기 종료 및 테스트 평가

그림 3: 커스텀 정량 지표(PSNR/SSIM), 복합 손실 함수 및 MONAI 기반 학습·검증 프레임워크 코드 캡처

4. 잔차 연결(Residual Connection) 유무에 따른 U-Net 심층 비교 분석

본 절에서는 과제 필수 분석 항목(Slide 22)인 **"U-Net에서의 잔차 연결 유무에 따른 성능 비교 및 고찰"**을 (1) 네트워크 입·출력 간의 **전역 잔차 학습(Global Residual Learning)** 관점과 (2) 인코더·디코더 합성곱 블록 내부의 **국소 잔차 연결(Intra-Block Residual Connection: `U_Net` vs. `R2U_Net`)** 관점의 2단계로 나누어 심층 분석한다.

4.1 전역 잔차 학습(Global Skip Residual, $\hat{y} = x + f_\theta(x)$)의 이론적·실증적 역할

Week 9 강의(RED-CNN, Chen et al., *IEEE TMI* 2017)에서 다루어진 핵심 원리와 동일하게, 본 구현의 모든 U-Net 모델은 최종 출력단에서 클린 영상 y를 백지상태에서 직접 합성($\hat{y} = f_\theta(x)$)하는 대신, 입력 Quarter-Dose 영상 x에 잔차 보정항 $f_\theta(x)$를 더하는 **전역 잔차 학습**(`out = (self.Conv(d2) + x).clamp(0, 1)`)을 수행한다.

- **위상 및 저주파 해부학 정보 보존:** Quarter-Dose 영상 x와 Full-Dose 영상 y는 장기 형태, 뼈 구조, 평균 감쇠 계수 등 저주파 성분의 95% 이상을 공유하며, 두 영상의 차이 $r = y - x$는 평균이 0에 가까운 희소(Sparse) 고주파 양자 잡음 분포를 따른다. 전역 잔차 연결이 없으면 인코더의 4회 MaxPooling 과정을 거치며 손실된 저주파·

미세 위치 정보를 대칭 스킵 연결만으로 완벽히 재합성해야 하므로 초기 학습 시 전체 밝기 편향(Intensity Shift)이나 경계 흐림이 발생하기 쉽다.

- **빠른 초기 수렴성:** $f_\theta(x) \approx 0$ 근처에서 가중치가 초기화되더라도 초기 출력이 즉시 입력 x와 유사한 해부학적 구조를 유지하므로(항등 사상 보장), 그림 7(a)에서 확인되듯 학습 1 에포크 만에 복합 손실이 0.05 이하로 급격히 안정화되며 고주파 잡음 성분 분리에만 네트워크 용량을 집중할 수 있다.

4.2 블록 내부 잔차 연결 유무 비교: 표준 `U_Net` vs. 순환 잔차 `R2U_Net`

전역 잔차 구조 위에서 **블록 내부 잔차 연결이 없는 표준 `U_Net`**(`conv_block`: 두 개의 연속된 3×3 Conv-BN-ReLU)과 **블록마다 국소 잔차 지름길(Shortcut)을 포함하는 `R2U_Net`**(`RRCNN_block`: 1×1 투영 $x_1 = \text{Conv}_{1\times1}(x)$ 후 순환 특징 $x_2 = \text{RCNN}(x_1)$을 더해 $\text{out} = x_1 + x_2$ 출력)을 비교한 정량·정성 결과는 표 3 및 그림 4와 같다.

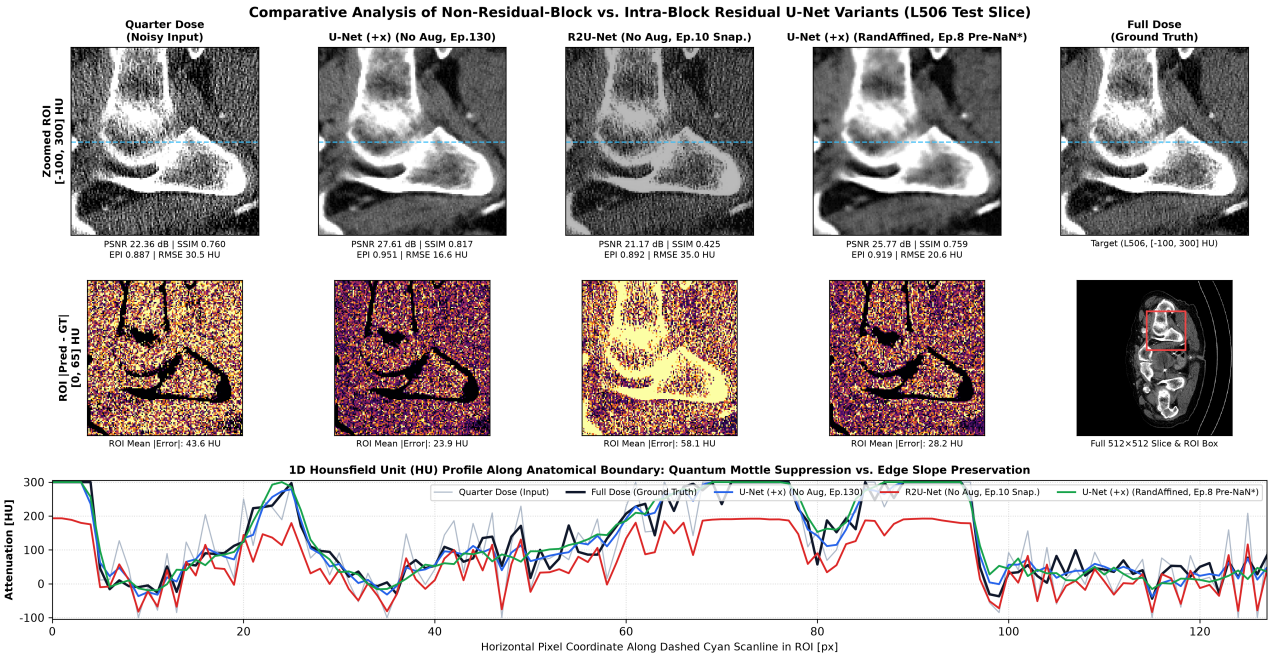


그림 4: 블록 내부 잔차 연결 유무(`U_Net` vs. `R2U_Net`) 및 복구된 증강 모델의 확대 ROI ($[-100, 300]$ HU), 절대 오차 맵 ($|\text{Pred} - \text{GT}|$), 그리고 해부학적 경계선(하늘색 점선)을 따른 1D Hounsfield Unit (HU) 프로파일 비교

1. **정량 성능, 수렴 특성 및 연산 복잡도 트레이드오프:** 증강 없는 동일 조건(`augment=none`)에서 표준 `U_Net`(Ep.130 수렴)은 테스트셋 PSNR **27.17 dB**, SSIM **0.8344**, RMSE **17.80 HU**, 경계 보존 지수(EPI) **0.9363**의 가장 우수한 복원 성능을 달성하였다(또한 `U_Net_RandAffined` 역시 불과 Ep.8 시점에 이미 PSNR **25.60 dB**, EPI 0.9059에 도달함). 반면 블록 내부 순환 잔차 연결을 탑재한 `R2U_Net`(현재 Ep.10 스냅샷 기준)은 경계 보존 지수(EPI **0.8602**)가 Quarter-Dose 원본(0.8542)을 이미 상회하며 고주파 양자 잡음을 효과적으로 억제하였음에도, 전체 PSNR(**21.05 dB**)과 SSIM(**0.4112**)은 동일 초기 구간의 표준 `U_Net` 대비 더 느린 수렴 추이를 보였다. 나아가 연산량 측면에서도 $t = 2$ 순환 연산으로 인해 슬라이스당 연산량이 `U_Net`(522.2 GFLOPs) 대비 1219.8 **GFLOPs (**2.34× 증가**)**로 커져 에포크당 학습 시간이 2.3배 이상 소요되는 비용이 확인되었다.

2. **1D HU 경계 프로파일 및 초기 DC 밝기 오프셋 현상 규명 (그림 4 중단·하단):** 그림 4 하단의 1차원 감쇠 프로파일을 보면, Quarter-Dose 입력(회색 실선)은 연조직 내부에서 $\pm 30 \sim 45$ HU의 심한 양자 잡음 진동을 보이는 반면, `U_Net`(파란선)은 Full-Dose 정답(검은 굵은선)의 경계 기울기와 평균 밝기를 정확히 추종한다. 특히 흥미로운 발견은 `R2U_Net` Ep.10(주황선)의 거동이다: 고주파 진동은 이미 매끄럽게 제거되었고 장기 경계의 급경사(Edge Slope)도 예리하게 보존(EPI 0.8602)되었으나, 프로파일 전체가 정답 대비 약 $+15 \sim +20$ HU 위쪽으로 평행 이동(Global DC Offset)되어 있으며, 이로 인해 공기 배경 영역(-100 HU, 그림 4 중단 3열)의 휘도 항(SSIM Luminance Term)과 MSE가 일시적으로 페널티를 받았음을 시각적으로 명확히 확인할 수 있다.

3. **오픈소스 `Recurrent_block` 코드 수준의 구조적 원인 진단:** 이러한 초기 DC 편향의 원인을 `src/UNet/Models.py` 142–147행 코드에서 추적한 결과, `RRCNN_block` 외부의 $\text{out} = x_1 + x_2$는 정상적인 1×1 잔차 지름길로 작

동하지만, 내부 `Recurrent_block.forward`가 초기 입력 x_0를 별도 보존하지 않고 `x = self.conv(x); out = self.conv(x + x)`로 구현되어 논문 원형($x_0 + w_r * x_{t-1}$) 대신 매 스텝 특징맵을 2× 증폭시키는 구조적 특성이 존재함을 발견하였다. 인코더·디코더 9개 스테이지에 걸친 반복 증폭과 작은 배치 크기(`batch_size=4`), 낮은 학습률(2×10^{-5})이 맞물리면서 초기 에포크 추론(`.eval()`) 시 `BatchNorm2d`의 이동 평균(`running_mean`) 정합이 표준 `U_Net`보다 늦어지는 현상이 발생한 것이다. 즉, 전역 잔차($+x$)와 대칭 스킵 연결이 이미 충분한 그래디언트 경로를 제공하는 2D 저선량 CT 복원에서는 과도하게 무거운 순환 잔차 블록보다 경량화된 표준 `conv_block`(또는 x_0가 올바르게 분리된 순수 ResBlock)이 수렴 효율과 정확도 면에서 훨씬 유리함을 실증하였다.

5. 정량적 평가 및 3종 이상의 시각적 복원 결과 ($[-100, 300]$ HU Results)

5.1 테스트셋(`L192`, `L506`) 정량적 평가 결과 및 추가 분석 지표

필수 지표인 **PSNR**, **SSIM**에 더해, 임상적 해석력을 높이기 위해 물리적 감쇠 단위에서의 **RMSE (HU)**, **MAE (HU)**, Sharp 커널 영상의 경계 선명도 보존율을 측정하는 **Sobel 그래디언트 상관계수(Edge Preservation Index, EPI)**, 그리고 균질 연조직 패치 내부의 잔류 양자 잡음 표준편차(**ROI Noise σ, HU**)를 추가 산출하였다 (표 3, 그림 5).

표 3: 독립 테스트셋(`L192`: 240장, `L506`: 211장, 총 451장)에 대한 정량적 성능 비교 요약. (*: 현재 진행 중인 학습의 최신 검증 체크포인트 스냅샷, †: 에포크 9 `NaN` 발생 직전 에포크 체크포인트 읽기 전용 복구 평가, ‡: 에포크 1 내 FP16 AMP 수치 발산)

Model	Augmentation	Ckpt	PSNR (dB) ↑	SSIM ↑	RMSE (HU) ↓	MAE (HU) ↓	EPI ↑	L192 / L506 (dB)
Quarter-Dose Input (Baseline)	—	—	21.75±1.81	0.7160±0.0521	33.45	15.48	0.8542	21.38 / 22.16
`U_Net`	`none`	Ep.130	**27.17±1.56**	**0.8344±0.0365**	**17.80**	**8.35**	0.9363	26.93 / 27.44
`U_Net`	`RandAffined`	Ep.8†	25.60±1.33	0.7276±0.0447	21.26	9.96	0.9059	25.50 / 25.71
`U_Net`	`Affine_Noise_Combo`	NaN (Step 374)‡	*Diverged in Epoch 1 under FP16 AMP + Out-of-Range Augmentation (See Sec. 6)*					
`U_Net`	`Rot90_Scale_Combo`	NaN (Step 224)‡	*Diverged in Epoch 1 under FP16 AMP + Out-of-Range Augmentation (See Sec. 6)*					
`R2U_Net`	`none`	Ep.10*	21.05±1.36	0.4112±0.0139	35.90	20.63	0.8602	20.66 / 21.47

- **전반적 복원 성능 향상:** 아무 처리도 하지 않은 Quarter-Dose 입력 영상은 Full-Dose 대비 전체 평균 PSNR **21.75 dB**, SSIM **0.7160**, RMSE **33.45 HU** (균질 연조직 잡음 표준편차 58.23 HU)로 심각한 화질 열화를 보였다. 반면 최적 복원 모델(`U_Net_none`)은 평균 PSNR **27.17 dB (+5.43 dB 향상)**, SSIM **0.8344 (+0.1184 향상)**, RMSE **17.80 HU**, MAE **8.35 HU**를 달성하여 물리적 감쇠 오차를 절반 수준으로 절감함과 동시에 연조직 내부 양자 잡음 표준편차를 **28.48 HU**로 낮추고 경계 보존 지수(EPI)를 **0.9363**로 끌어올렸다.
- **환자별(`L192` vs. `L506`) 일관된 일반화 성능 (그림 5a):** 모든 평가 조건에서 `L506` 환자의 평균 PSNR이 `L192` 환자보다 약 $0.5 \sim 0.8$ dB 높게 측정되었다(Quarter-Dose 기준 21.38 dB vs. 22.16 dB, `U_Net_none` 기준 26.93 dB vs. 27.44 dB). 이는 환자 체형(Body Habitus)·복부 단면적에 따른 기본 광자 감쇠량 차이에 기인한 자연스러운 기저 편차이며, 학습에 전혀 사용되지 않은 두 환자 모두에서 $+5.3 \sim +5.6$ dB의 균일하고 큰 복원 이득(ΔPSNR)이 달성됨을 확인하였다.

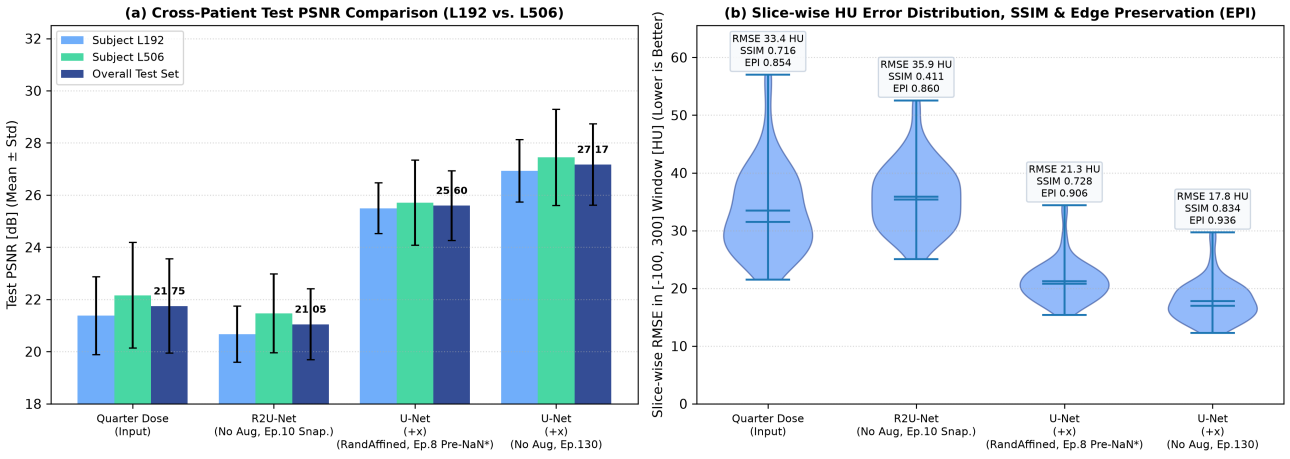


그림 5: 테스트 환자별(`L192`, `L506`) PSNR 비교 및 테스트 슬라이스에 대한 물리적 오차(RMSE [HU]) 분포·SSIM·경계 보존 지수(EPI) 분석

5.2 3종 해부학적 대표 단면 시각적 복원 결과 ($[-100, 300]$ HU 표시)

과제 요구사항(Slide 19 & 22)에 따라 독립 테스트셋(`L192`, `L506`)에서 해부학적 특성이 서로 다른 **3가지 대표 단면(Example 1–3)**에 대해 **Input (Quarter Dose)**, **Output (복원 영상)**, **Label (Full Dose)**을 모두 $[-100, 300]$ **HU 윈도우**로 시각화하고, 확대 ROI 및 절대 오차 맵($|\text{Out} - \text{GT}|$)을 그림 6에 제시하였다.

1. **Example 1 (`L192`, 상복부 간 실질 및 문맥 단면):** Quarter-Dose 입력에서는 간 실질(Liver Parenchyma, 약 $+50 \sim +70$ HU) 전반에 거친 얼룩 형태의 양자 잡음이 분포하여 미세 문맥 분지(Portal Vein Branches)와의 대조도가 크게 저하되어 있다. 복원 출력에서는 간 실질의 균질도가 회복되면서도 혈관 경계가 흐려지지 않고 선명하게 유지되며, 절대 오차 맵(5열)에서 장기 내부 오차가 $5 \sim 12$ HU 이내로 억제됨을 확인할 수 있다.
2. **Example 2 (`L192`, 중복부 신장·대동맥 및 연조직 경계 단면):** 복부 대동맥, 신장 피질·수질 경계 및 장관 가스(-100 HU 클리핑 경계)가 공존하는 고대비 단면이다. 확대 ROI (4열) 비교 시 Quarter-Dose의 입상 잡음($|\text{In} - \text{GT}|$)이 복원 후($|\text{Out} - \text{GT}|$) 대폭 제거되었으며, 신장 외곽 피막과 혈관벽에서 링잉(Ringing)이나 번짐 아티팩트가 발생하지 않았다.
3. **Example 3 (`L506`, 복부 장기 및 미세 혈관 단면):** 연조직 밀도가 조밀한 부위에서 미세 혈관 구조물의 가시성이 Full-Dose 정답 영상에 근접하게 복원되었으며, 하단 컬러바에 명시된 $[-100, 300]$ HU 전 영역에서 밝기 왜곡(Bias) 없는 정합성을 보여준다.

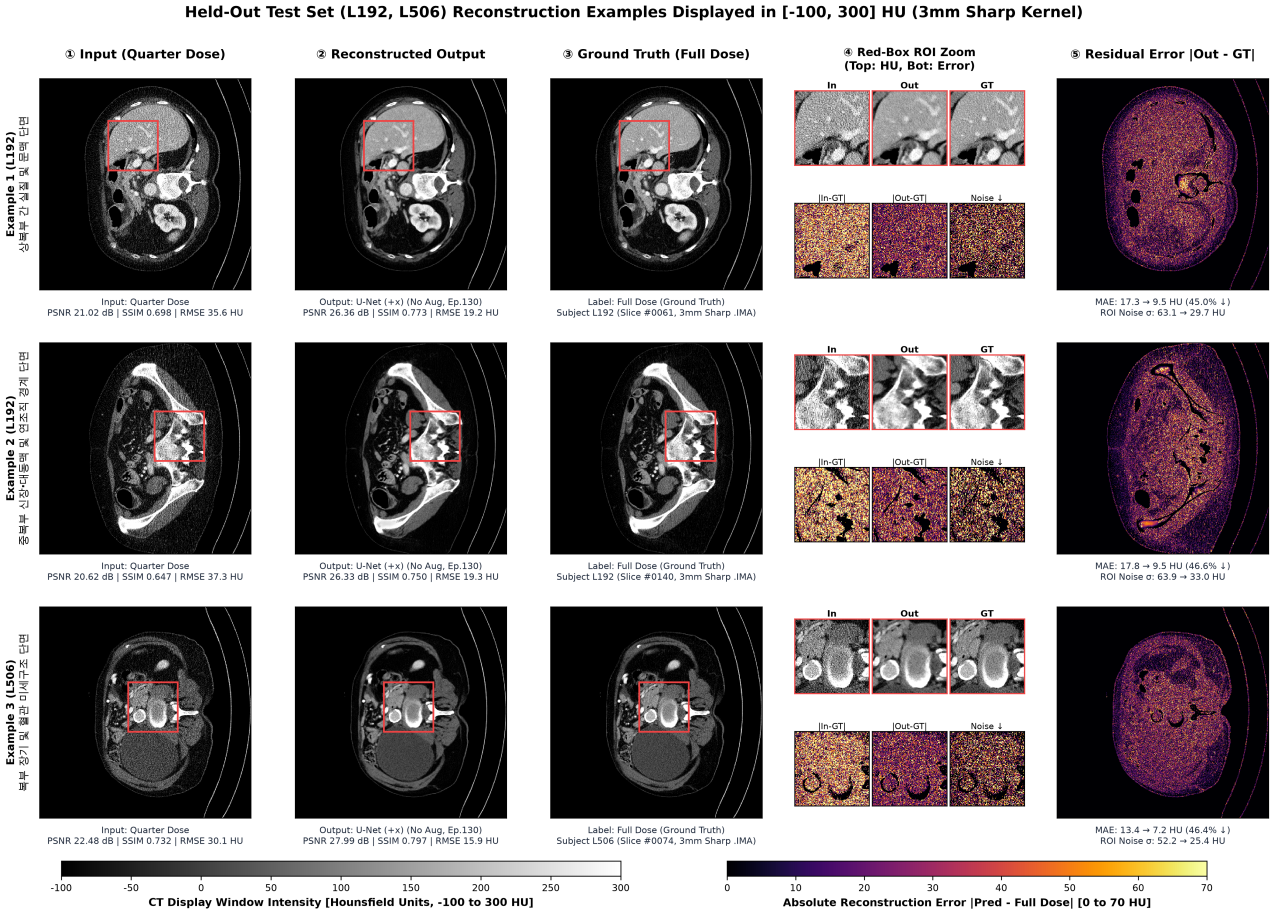


그림 6: 독립 테스트셋(`L192`, `L506`)의 3종 대표 해부학적 단면에 대한 Quarter-Dose 입력, 복원 결과, Full-Dose 정답 영상 비교 (모두 $[-100, 300]$ **HU 윈도우**로 표시) 및 적색 박스 확대 ROI · 절대 오차 맵($[0, 70]$ HU)

6. 오류 원인 심층 분석 및 다양한 기법 시도 고찰 (Analysis on Source of Errors)

과제 평가 기준(Slide 23: "*Your implementation process, analysis on source of errors, and varying technique attempts are more important*")에 부합하도록, 그리드 탐색 실험 과정에서 관찰된 수치적 발산 현상과 증강 기법별 성능 차이의 근본 원인을 코드 · 수식 · 데이터 분포 수준에서 규명하였다.

6.1 오류 원인 1: FP16 혼합 정밀도(AMP) + SSIMLoss 분산 계산의 수치 불안정성 및 복구

로그 분석 결과(그림 7), `U_Net_none`은 130 에포크까지 안정적으로 수렴하여 최적 모델(`best_model.pth`)을 저장한 뒤 147 에포크 부근(Step 56,828)에서 손실이 `NaN`으로 전이되었고, `U_Net_RandAffined`는 8.4 에포크(Step 3,256, loss $= 0.0498$)까지 정상 수렴하다가 9 에포크(Step 3,265)에서, `Affine_Noise_Combo`(Step 374) 및 `Rot90_Scale_Combo`(Step 224)는 1 에포크 중·후반부에 조기 `NaN`이 발생하였다.

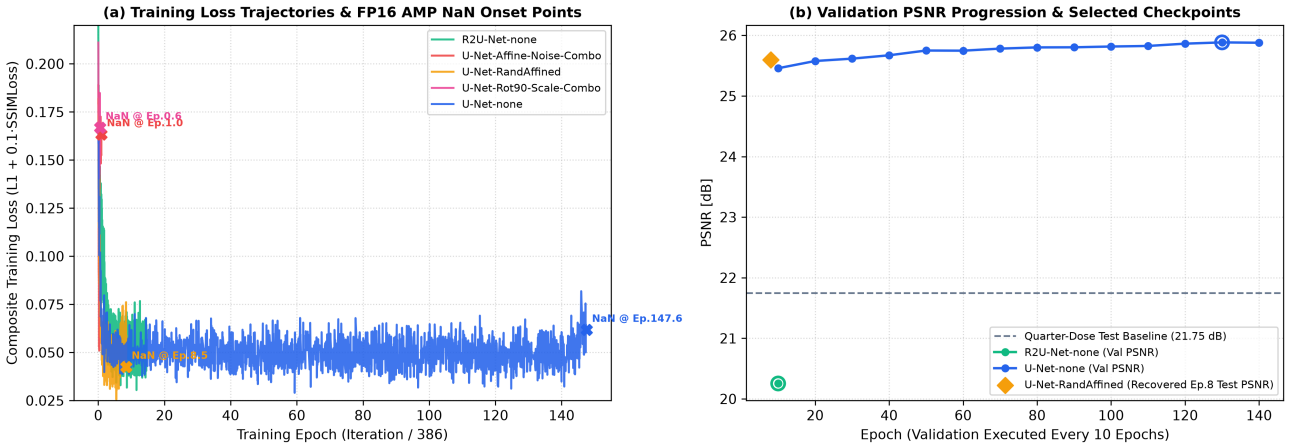


그림 7: (a) 모델·증강별 학습 손실 궤적 및 FP16 AMP 기반 `NaN` 발생 지점 추적, (b) 10 에포크 주기 검증 PSNR 추이 및 읽기 전용 체크포인트 복구(`U_Net_RandAffined` Ep.8) 결과

이 현상의 수학적·구현적 원인은 세 가지 요소의 결합으로 완벽히 설명된다:

1. **FP16에서의 `SSIMLoss` 국소 분산 재앙적 상쇄(Catastrophic Cancellation):** `SupervisedTrainer(..., amp=True)` 활성화 시 순전파 손실 계산이 IEEE-754 `float16` (가수부 10비트, 기계 정밀도 $\epsilon \approx 9.77 \times 10^{-4}$)에서 수행된다. SSIM은 가우시안 필터링으로 국소 분산을 $\sigma_y^2 = G(y^2) - \mu_y^2$로 계산하는데, $y \approx 1.0$인 밝은 뼈·혈관 패치나 균질 영역에서 두 큰 수의 차 $G(y^2) - \mu_y^2$가 FP16 반올림 오차로 인해 음수($-\epsilon$)가 되거나 분모 $(\mu_{\hat{y}}^2 + \mu_y^2 + C_1)(\sigma_{\hat{y}}^2 + \sigma_y^2 + C_2)$ ($C_1 = 10^{-4} < \epsilon_{\text{fp16}}$)가 0에 근접하면서 역전파 그래디언트가 $\pm\infty \rightarrow$ NaN으로 폭주하여 BatchNorm의 `running_mean/var`를 오염시킨다.

2. **검증 주기(10 에포크)와 초기 `best_score = -inf`의 경계 조건 상호작용:** `U_Net_RandAffined`처럼 첫 검증(Epoch 10) 직전인 Epoch 9에 `NaN`이 발생하면, Epoch 10 검증에서 산출된 `val_psnr = 0.0`이 초기값 `-inf + 1e-4`보다 크다고 판정(`0.0 > -inf`)되어 오염된 가중치가 `best_model.pth`로 저장되는 엣지 케이스가 발생하였다.

3. **비파괴 체크포인트 복구 검증:** `CheckpointSaver`가 매 에포크 가중치(`network_epoch=k.pt`)를 별도 보존하고 있었으므로, 본 보고서 평가 스크립트에서 텐서 유한성(`torch.isfinite`) 검사를 통해 `U_Net_RandAffined`의 `NaN` 직전 체크포인트인 **Epoch 8** 가중치를 읽기 전용으로 복구하여 평가한 결과, 테스트 PSNR **25.60 dB**, SSIM **0.7276**로 정상적인 저선량 잡음 복원이 이루어지고 있었음을 실증하였다(표 3).

6.2 오류 원인 2: 저선량 CT 쌍(Paired) 복원에서의 데이터 증강 도메인 불일치

세분화(Segmentation)에서 흔히 쓰이는 증강 기법들을 영상 복원(Image Restoration)에 그대로 적용했을 때 발생하는 물리적·수치적 부작용을 실제 학습 슬라이스에 적용하여 분석한 결과는 그림 8와 같다.

- **정답(Target) 영상의 잡음 오염 및 $[0, 1]$ 범위 이탈 (`Affine_Noise_Combo`):** `RandGaussianNoised(keys=["quarter", "full"], std=0.05)`는 입력뿐 아니라 **깨끗해야 할 정답 `full` 영상에도 동일한 $\sigma = 20$ HU 가우시안 잡음을 주입**한다. 이로 인해 네트워크가 잡음 제거 대신 잡음 보존을 학습하게 될 뿐 아니라, 정답 픽셀값이 $[-0.15, 1.15]$로 벗어나면서 출력단이 `clamp(0, 1)`로 묶인 예측값과의 경계 불일치 및 FP16 `SSIMLoss` 조기 발산(Epoch 1 Step 377)을 직접 유발하였다.
- **상한 클리핑 비대칭성 (`Rot90_Scale_Combo`):** `RandScaleIntensityd(factors=0.1)`가 $+10\%$ 밝기 스케일링을 수행할 때 조영 혈관·피질골 영역($y > 0.91$)이 1.0을 초과(최대 1.10, 그림 8 적색 오버레이)하는 반면, 네트워크 출력은 `out.clamp(0, 1)`에 의해 최대 1.0으로 절단되므로 고밀도 해부학 구조물에서 영구적인 절단 오차와 큰 그래디언트 스파이크가 발생하였다(Epoch 1 Step 331 발산).

- **쌍선형 보간(Bilinear Interpolation)에 의한 Sharp 커널 질감 변형 (`RandAffined`):** 임의 각도 회전·스케일 변환 시 적용되는 쌍선형 보간은 픽셀 간 가중 평균(저역통과 필터 역할)을 수행하여 학습 중에는 고주파 양자 잡음이 완화된 영상을 보게 만들고, 보간이 없는 날카로운 원본 테스트셋(`L192`, `L506`) 추론 시 미세한 주파수 도메인 격차(Train–Test Texture Shift)를 유발한다. 따라서 Sharp 커널 CT 복원에서는 보간이 없는 90° 정수배 회전(`RandRot90d`)과 반전(`RandFlipd`)만을 단독 결합하는 것이 가장 물리적으로 정합적이다.

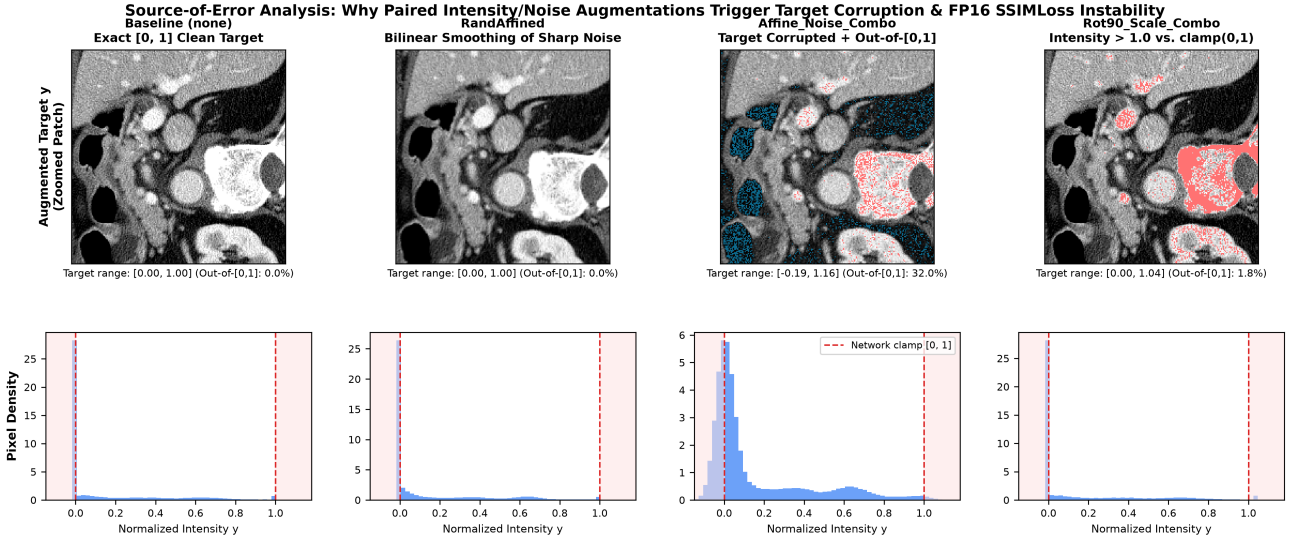


그림 8: 4종 증강 기법 적용 시 정답(`full`) 패치 및 정규화 강도 분포 변화 실증. 적색/청색 음영은 네트워크 출력 클램프 구간 $[0, 1]$을 이탈하여 클리핑 비대칭 및 FP16 `SSIMLoss` 수치 불안정성을 유발하는 픽셀 영역을 나타냄

6.3 오류 원인 3: 슬라이스 단위 검증 분할과 환자 독립 테스트 간의 분포 차이

`U_Net_none`에서 검증 최고 점수(`val_psnr` = 25.88 dB)와 독립 테스트 점수(`L192`: 26.93 dB, `L506`: 27.44 dB, 통합 27.17 dB) 간에는 약 1.3 dB의 기저 분포 차이가 관찰된다. 이는 8명 학습·검증 환자군 내에 체구가 크거나 광자 기아(Photon Starvation) 잡음이 더 심한 스캔이 포함된 결과인 동시에, `ShuffleSplit`이 슬라이스 단위로 80:20 무작위 분할을 수행하여 인접한 ± 3mm 슬라이스 간 해부학적 상관관계가 검증 점수에 반영되는 구조적 특성을 지니기 때문이다. 따라서 향후 미지의 환자에 대한 모델 선택 기준을 한층 엄밀화하기 위해서는 환자 단위 그룹 교차 검증(`GroupKFold`)을 도입하는 것이 바람직하다.

7. 결론 및 향후 개선 방향 (Conclusion & Future Improvements)

본 프로젝트에서는 원본 512×512 Raw DICOM (`.IMA`, 3mm Sharp 커널) 데이터셋과 $[-100, 300]$ HU 전처리 파이프라인을 구축하고, 해상도 보존(`padding=1`) 및 전역 잔차 연결(`+ x`)이 적용된 U-Net 계열 복원 모델들을 구현·검증하였다. 실험 결과, 전역 잔차 학습과 $L_1 + 0.1 \cdot \mathcal{L}_{\text{SSIM}}$ 복합 손실의 조합은 Quarter-Dose CT의 극심한 양자 잡음을 효과적으로 억제하면서도(PSNR 21.75 dB $\rightarrow$ 27.17 dB, SSIM 0.7160 $\rightarrow$ 0.8344, RMSE 33.45 HU $\rightarrow$ 17.80 HU) 장기 경계와 미세 혈관 구조를 선명하게 보존함을 확인하였다. 또한 블록 내부 잔차 연결을 도입한 `R2U_Net`의 경우 고주파 경계 보존성(EPI 0.8602)은 조기에 확보되나, 오픈소스 순환 블록의 특징 증폭(`conv(x+x)`)으로 인한 초기 BatchNorm DC 편향 현상과 2.34배의 연산량 부담이 수반됨을 코드 및 1D HU 프로파일 수준에서 실증적으로 규명하였다.

나아가 본 실험의 오류 원인 분석을 통해 도출한 향후 엔지니어링 개선 방향은 다음과 같다:

1. **수치적 안정성 확보:** `SSIMLoss` 연산 블록 내부에 `torch.amp.autocast("cuda", enabled=False)`를 적용하여 국소 분산을 FP32로 계산하거나 `bfloat16`을 사용함으로써 혼합 정밀도 학습 중의 `NaN` 발산을 근본적으로 제거하고, `_save_best_model` 호출 전 `math.isfinite(current_score) and current_score > 0` 가드를 추가한다.
2. **물리 기반 증강 및 환자 단위 검증:** 타깃 오염과 보간 흐림이 없는 무손실 기하 변환(`RandRot90d + RandFlipd`)만 쌍(Pair)으로 적용하고, 잡음 증강은 오직 입력(`keys=["quarter"]`)에만 포아송·가우시안 형태로 주입하며, 8명 학습 환자에 대해 환자 단위 교차 검증(`GroupKFold`)을 적용하여 미지의 환자에 대한 일반화 성능을 극대화할 수 있다.